

magnitude slower. On the other hand, with kernel-level threads, having a thread block on I/O does not suspend the entire process as it does with user-level threads.

Since the kernel knows that switching from a thread in process *A* to a thread in process *B* is more expensive than running a second thread in process *A* (due to having to change the memory map and having the memory cache spoiled), it can take this information into account when making a decision. For example, given two threads that are otherwise equally important, with one of them belonging to the same process as a thread that just blocked and one belonging to a different process, preference could be given to the former.

Another important factor is that user-level threads can employ an application-specific thread scheduler. Consider, for example, the Web server of Fig. 2-8. Suppose that a worker thread has just blocked and the dispatcher thread and two worker threads are ready. Who should run next? The run-time system, knowing what all the threads do, can easily pick the dispatcher to run next, so that it can start another worker running. This strategy maximizes the amount of parallelism in an environment where workers frequently block on disk I/O. With kernel-level threads, the kernel would never know what each thread did (although they could be assigned different priorities). In general, however, application-specific thread schedulers can tune an application better than the kernel can.

2.6 RESEARCH ON PROCESSES AND THREADS

In Chap. 1, we looked at some of the current research in operating system structure. In this and subsequent chapters, we will look at more narrowly focused research, starting with processes. As will become clear in time, some subjects are much more settled than others. Most of the research tends to be on the new topics, rather than ones that have been around for decades.

The concept of a process is an example of something that is fairly well settled. Almost every system has some notion of a process as a container for grouping together related resources such as an address space, threads, open files, protection permissions, and so on. Different systems do the grouping slightly differently, but these are just engineering differences. The basic idea is not very controversial any more, and there is little new research on the subject of processes.

Threads are a newer idea than processes, but they, too, have been chewed over quite a bit. Still, the occasional paper about threads appears from time to time, for example, about core-aware thread management (Qin et al., 2019), or on how well modern operating systems like Linux scale with many threads and many cores (Boyd-Wickizer, 2010).

Additionally, there is a lot of work trying to prove things do not break in the presence of concurrency, for instance in file systems (Chajed et al., 2019; and Zou et al., 2019) and other services (Setty et al., 2018; and Li et al., 2019). This is important work as researchers have shown that concurrency bugs are unfortunately

extremely common (Li et al., 2019). As we have seen, locking is not just hard, but also expensive and operating systems have adopted RCU to avoid locking altogether (McKenney et al., 2013).

An active research area deals with recording and replaying a process' execution (Viennot et al., 2013). Replaying helps developers track down hard-to-find bugs and security experts to investigate incidents.

Speaking of security, a major event in 2018 was the disclosure of a series of very serious security vulnerabilities in modern CPUs. They required changes everywhere: hardware, firmware, operating system, and even applications. For this chapter, the scheduling implications are especially relevant. For instance, Windows adopted a scheduling algorithm to prevent code in different security domains from sharing the same processor core (Microsoft, 2018).

Scheduling (both uniprocessor and multiprocessor) in general is still a topic near and dear to the heart of some researchers. Some topics being researched include scheduling in clusters for deep learning (Xiao et al., 2018), scheduling for microservices (Sriraman, 2018), and schedulability (Yang et al., 2018). All in all, processes, threads, and scheduling are not hot topics for research as they once were. The research has moved on to topics like power management, virtualization, clouds, and security.

2.7 SUMMARY

To hide the effects of interrupts, operating systems provide a conceptual model consisting of sequential processes running in parallel. Processes can be created and terminated dynamically. Each process has its own address space.

For some applications, it is useful to have multiple threads of control within a single process. These threads are scheduled independently and each one has its own stack, but all the threads in a process share a common address space. Threads can be implemented in user space or in the kernel.

Alternatively, high-throughput servers may opt for an event-driven model instead. Here, the server operates as a finite state machine that responds to events and interacts with the operating system using non-blocking system calls.

Processes can synchronize with one another using synchronization and interprocess communication primitives, for example, semaphores, monitors, or messages. These primitives are used to ensure that no two processes are ever in their critical regions at the same time, a situation that leads to chaos. A process can be running, runnable, or blocked and can change state when it or another process executes one of the interprocess communication primitives. Interthread communication is similar.

A great many scheduling algorithms have been studied. Some of these are primarily used for batch systems, such as shortest-job-first scheduling. Others are common in both batch systems and interactive systems. These algorithms include